

Assignment 2: Fitting and Alignment

220268A - Jayasundara A.J.M.K.K.B.

Link to GitHub repository - <https://github.com/keshj/Computer-Vision-and-Image-Processing>

Question 1



Sigma range used: [1.0. 10.0] at 0.25 increments, resulting in 37 scales. The largest blob was detected at $\sigma = 9.50$, and the radius is 13.44 pixels. This blob had a response strength of 0.1680.

Creating the scale space and applying the filter:

```
1 for i, sigma in enumerate(sigmas):
2     hw = round(3 * sigma)
3     X, Y = np.meshgrid(np.arange(-hw, hw + 1), np.arange(-hw, hw + 1))
4     LoG = (X**2 + Y**2 - 2 * sigma**2) * np.exp(-(X**2 + Y**2) / (2 * sigma**2))
5     LoG /= 2 * np.pi * sigma**6
6     LoG *= sigma**2
7     im_Log = cv.filter2D(im_gray, -1, LoG)
8     scale_space[:, :, i] = np.abs(im_Log)
```

Scale space extrema detection:

```
1 for scale_idx in range(1, num_scales - 1):
2     for i in range(1, h - 1):
3         for j in range(1, w - 1):
4             center_val = current_scale[i, j]
5             if center_val < abs_threshold: continue
6             # Check 3x3 neighborhood
7             if center_val != np.max(current_scale[i-1:i+2, j-1:j+2]): continue
8             # Check scale neighbors
9             if center_val <= prev_scale[i, j] or center_val <= next_scale[i, j]: continue
10         blobs.append((j, i, sigma, center_val))
```

Important parts from the code for non maxima suppression:

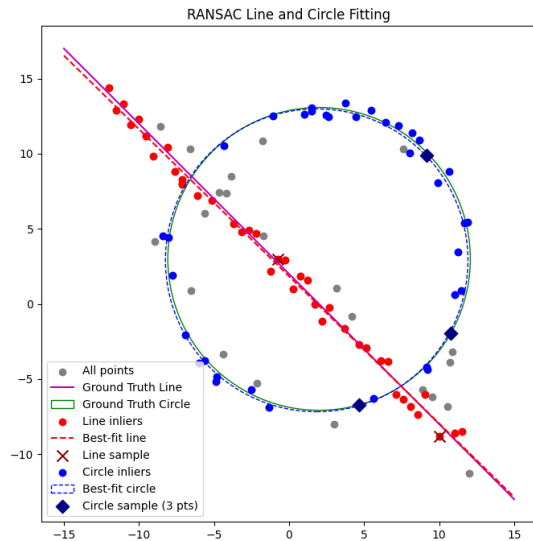
```
1 blobs_sorted = sorted(blobs, key=lambda b: b[3], reverse=True)
2 radius = np.sqrt(2) * sigma1 # Computing the radius from sigma
3 distance = np.sqrt((y1 - y2)**2 + (x1 - x2)**2) # Overlap calculation
```

```

4 sum_radii = radius1 + radius2
5 overlap = 1 - (distance / sum_radii)
6 if overlap < overlap_threshold:
7     remaining.append(blob)

```

Question 2



a) For the RANSAC line estimation, **2 random points** are used in each iteration, and best-fit lines were found using the Total Least Squares method. The resulting line parameters were $(a, b, d) = (0.69, 0.71, 1.32)$, getting the **line normal magnitude to 0.99**. And RANSAC iterations = 2000, error threshold = 0.7, minimum inliers required was set to 35.

b) In the RANSAC circle estimation, **3 random points** are used in each iteration. The algorithm was run for 500 iterations, with error threshold 0.7 and requiring atleast 35 inliers. The best fit circle was found with parameters $(xc, yc, r) = (1.898, 2.67, 10.12)$.

```

def fit_line_TLS(points):
    x_mean, y_mean = np.mean(points, axis=0)
    U = points - np.array([x_mean, y_mean])
    _, _, Vt = linalg.svd(np.transpose(U)@U) # Single Value Decomposition
    a, b = Vt[-1]
    norm = np.sqrt(a**2 + b**2)
    a, b = a / norm, b / norm # unit normal vector
    d = a * x_mean + b * y_mean
    return a, b, d

def ransac_line(X, n_iters=2000, threshold=0.7, min_inliers=35):
    best_inliers = [] # largest set of inliers
    best_param = None # best line parameters

    for _ in range(n_iters):
        idx = np.random.choice(X.shape[0], 2, replace=False) # pick 2 points
        sample = X[idx]
        a, b, d = fit_line_TLS(sample)
        # Apply threshold to find inliers
        distances = np.abs(a * X[:, 0] + b * X[:, 1] - d)
        inliers = np.where(distances < threshold)[0]

        if len(inliers) > len(best_inliers): # choose the model with more inliers
            best_inliers = inliers
            best_param = (a, b, d)

    if best_param is not None and len(best_inliers) > min_inliers:
        a, b, d = fit_line_TLS(X[best_inliers])
        return a, b, d, best_inliers

    return None, None, None, [] # no line found

```

(a) Fitting the line

```

def circle_from_3pts(pts):
    A = np.matmul([2 * (pts[1] - pts[0]), 2 * (pts[2] - pts[0])]).reshape(2, 2)
    b = np.array([
        np.dot(pts[1], pts[1]) - np.dot(pts[0], pts[0]),
        np.dot(pts[2], pts[2]) - np.dot(pts[0], pts[0])
    ])
    # If A is singular, the points are collinear, so we cannot fit a circle
    center = np.linalg.solve(A, b)
    r = np.linalg.norm(center - pts[0])
    return center[0], center[1], r

def ransac_circle(X, n_iter=500, threshold=0.6, min_inliers=35):
    best_inliers = []
    best_params = None

    for _ in range(n_iter):
        idx = np.random.choice(len(X), 3, replace=False)
        sample = X[idx]
        # Skip nearly collinear samples, using the area of the triangle they form
        area = np.abs(np.cross(sample[1]-sample[0], sample[2]-sample[0]))
        if area < 1e-2:
            continue
        try:
            xc, yc, r = circle_from_3pts(sample)
        except np.linalg.LinAlgError: # due to collinearity
            continue
        err = np.abs(np.sqrt((X[:, 0] - xc)**2 + (X[:, 1] - yc)**2) - r)
        inliers = np.where(err < threshold)[0]
        if len(inliers) > len(best_inliers):
            best_inliers = inliers
            best_params = (xc, yc, r)

    if best_params is not None and len(best_inliers) > min_inliers:
        xc, yc, r = circle_from_3pts(X[best_inliers][:3]) # refit using first 3 inliers
        best_params = (xc, yc, r)

    return best_params, best_inliers

```

(b) Fitting the circle

d) If we fit the circle first, it would likely give us worse results. This happens because a circle can sometimes fit straight line segments quite well (by having a very large radius). The circle might mistakenly identify many of the line points as its own inliers. Then when we go to fit the line afterward, we'd be left with fewer true line points to work with, making our line fit much less accurate.

Question 3

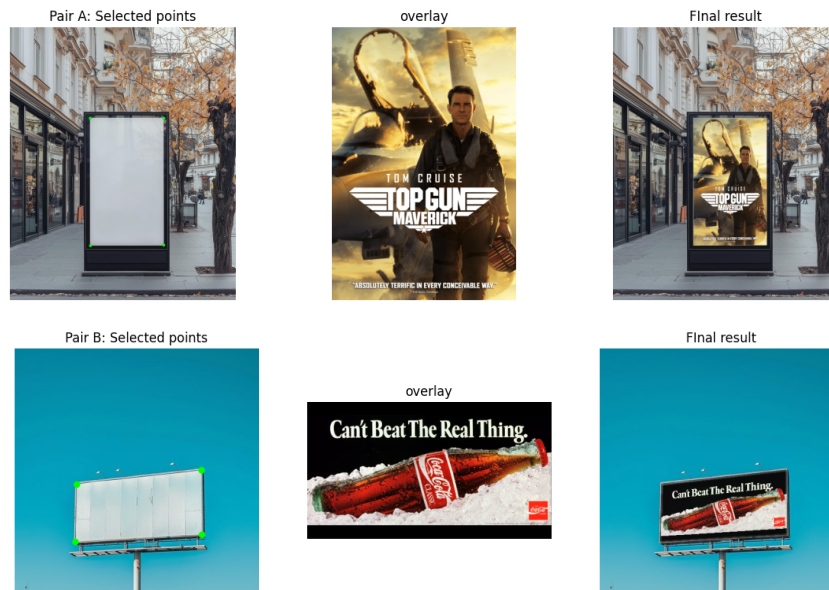


Figure 2: Selected points (green dots), overlay image and the final blended result.

The following code shows important parts of the code. The four points on the base image were selected by mouse clicks.

```
1 def compute_homography(src_points, dst_points):
2     n = len(src_points)
3     A = []
4     for i in range(n):
5         x, y = src_points[i]
6         x_p, y_p = dst_points[i]
7         A.append([0, 0, 0, -x, -y, -1, y_p*x, y_p*y, y_p])
8         A.append([x, y, 1, 0, 0, 0, -x_p*x, -x_p*y, -x_p])
9     A = np.array(A)
10    U, S, Vt = np.linalg.svd(A) # Solve using SVD
11    h = Vt[-1, :]
12    H = h.reshape(3, 3)
13    H = H / H[2, 2]
14    return H
15
16 def warp_image(image, H, output_shape):
17     h, w = output_shape
18     warped = cv2.warpPerspective(image, H, (w, h), flags=cv2.INTER_LINEAR)
19     mask = cv2.cvtColor(warped, cv2.COLOR_BGR2GRAY)
20     mask = (mask > 0).astype(np.uint8) * 255
21     return warped, mask
```

Question 4

To find matches between images, the SIFT algorithm was used, with tuned parameters **nFeatures**, **nOctaveLayers**, etc. to capture a higher number of matches since the two images have a large viewpoint difference. A Brute-Force matcher was used to compare the descriptors, and Low's ratio test is applied to eliminate poor matches.

```
1 def detect_and_match_sift(img1_gray, img5_gray):
2     cv2.SIFT_create(nfeatures=0, nOctaveLayers=15, contrastThreshold=0.01,
3                     edgeThreshold=15, sigma=1.6) # Create SIFT detector
4     # Detect keypoints and compute descriptors
5     kp1, desc1 = sift.detectAndCompute(img1_gray, None)
```

```

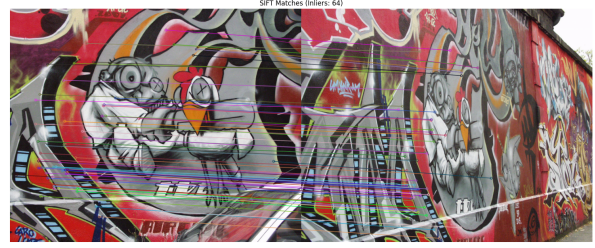
5  kp5, desc5 = sift.detectAndCompute(img5_gray, None)
6  bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
7  matches = bf.knnMatch(desc1, desc5, k=2)
8  good_matches = []
9  for m, n in matches:
10     if m.distance < 0.75 * n.distance: good_matches.append(m)
11  return kp1, kp5, good_matches

1 def compute_homography_ransac(src_pts, dst_pts, thresh=5, max_iters=2000):
2     best_H = None, max_inliers = 0
3     n_points = src_pts.shape[0]
4     for _ in range(max_iters):
5         idx = np.random.choice(n_points, 4, replace=False)
6         src_sample, dst_sample = src_pts[idx], dst_pts[idx]
7         H = compute_homography_dlt(src_sample, dst_sample)
8         src_homo = np.hstack([src_pts, np.ones((n_points, 1))])
9         dst_pred = (H @ src_homo.T).T
10        dst_pred[:, :2] / dst_pred[:, 2:] # Normalize
11        errors = np.sqrt(np.sum((dst_pts - dst_pred)**2, axis=1))
12        inliers = np.where(errors < thresh)[0] # Count inliers
13        if len(inliers) > max_inliers:
14            max_inliers = len(inliers), best_H = H
15    if len(best_inliers) >= 4: # Recompute H using all inliers
16        best_H = compute_homography_dlt(src_pts[best_inliers], dst_pts[
17        best_inliers])
18    return best_H, best_inliers

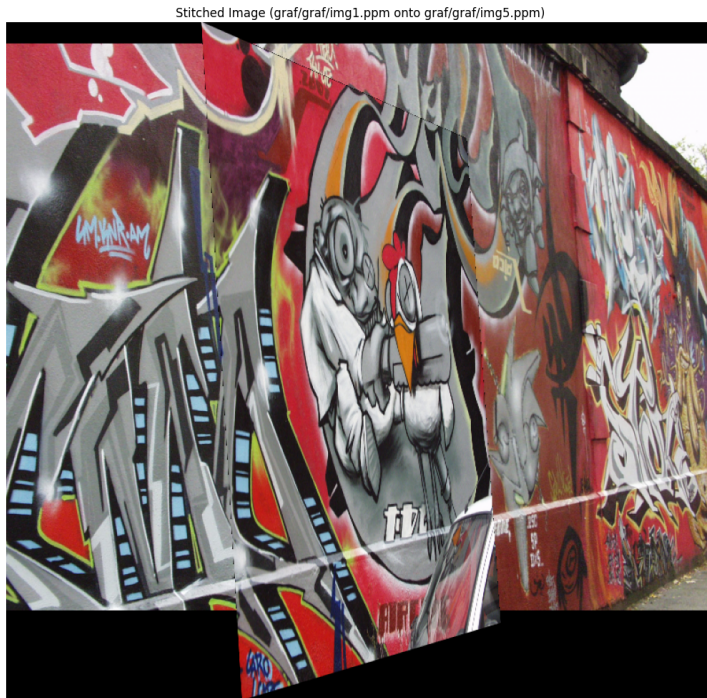
```



(a) SIFT matches (Total = 352)



(b) RANSAC Inliers = 64



The homography matrix required to warp img1 onto img5 was calculated using the RANSAC algorithm and the direct linear transform method. Threshold for selecting inliers was chosen by testing with trial and error. After **10000 iterations** algorithm picks the homography with the most inliers, which was **64 inliers**. The **computed matrix** is very close to the one provided in the dataset with an **absolute difference of 1.095**.

Figure 4: Result of the stitching.